Exploring and Modeling Team Competencies in Learning Engineering through Agentic Simulation

William Gray-Roncal[1] and Jodi Lis[2]

[1] Johns Hopkins University, School of Education; wgr@jhu.edu
[2] IEEE ICICLE; jodillis@gmail.com

Abstract. Learning engineering depends on teams whose expertise shapes decisions about learners and systems. This paper presents a configurable agentic simulation of that design process. The learning engineering flower provides eight starting expertise domains. Users can change team membership, responsibilities, and operating conditions, then inspect and replay the resulting design records, executable artifacts, and decisions. A scorecard examines human flourishing support, system performance, and learning. Three fictional cases show inconsistent feedback, compromised measurement, and unresolved operating assumptions; a fourth provides a scenario for further work. Software execution checks and reviews tied to source evidence separate observed artifact behavior from claims that require evaluation with real learners and practitioners. The tool and examples support investigation of team contributions and the development of hypotheses for empirical study.

Keywords: learning engineering; team competencies; agentic simulation; design evaluation

Introduction

Learning engineering combines expertise around a shared challenge, with team composition changing as the work develops (Ali et al., 2025). The practical question is how those contributions shape a solution: what learners can do, how people experience the intervention, and whether the wider system can operate it. Having a specialty represented does not ensure its concerns reach the design, implementation, or evaluation.

Agentic approaches already generate interactive learning experiences and training resources (Gröner et al., 2026; Ryng & Suhr, 2026). Here, users can change a team's composition and inspect its work across a complete design cycle. They can examine which contributions reach the solution, where responsibilities overlap, and what the team overlooks or discovers. Modeling can expose assumptions and guide inquiry without forecasting human behavior (Epstein, 2008). The software and worked examples provide a way to develop questions that learners and practitioners can investigate.

Configurable Teams and a Complete Design Process

The default team starts directly from the learning engineering flower's eight expertise areas (Table 1; Ali et al., 2025). The flower places shared understanding at the center and connects the team to learners and other stakeholders. Every team member has responsibility for those concerns. Assigning that responsibility does not substitute for involving people. The

instructions require members to distinguish supplied case facts, assumptions, and information still needed from learners or stakeholders.

The preset assigns one member to each area and a coordinator. Users can assign multiple domains to one member or share a domain across several members. Identity, competency instructions, behavioral settings, authority, and stage participation remain separate. The flower supplies an initial organization for the team; the editable instructions express how each area might contribute in the simulation. They are not validated measures of professional competence. The domain packages have no crosswalk to a separate competency framework.

Table 1. *Flower domains and illustrative contributions*

Source domain	Requested contribution in this simulation
Learning Sciences	Learning mechanisms, prior knowledge, practice, retention, and transfer
Learning Experience Design	Learner journeys, activity structure, feedback, and revision
Data Science	Data quality, analysis, uncertainty, and model assumptions
Assessment, Measurement, Evaluation	Constructs, measures, comparisons, and limits of inference
Learning Environment Engineering	Operating environment, dependencies, access, and recovery
Software Engineering	Executable behavior, state management, testing, and maintenance
Subject-Matter Expertise	Content accuracy, authentic tasks, and consequential errors
Education and Training Professionals	Facilitation, learner support, adoption, and delivery conditions

The process covers Understand, Map, Design, Build, Instrument, Experiment, Evaluate, and Refine. The flower preset schedules forty calls: four contributions per domain and eight coordinator decisions. Members contribute to shared stage records; the coordinator integrates proposals and records decisions. The resulting solution includes the challenge model, design rationale, executable resource, instrumentation and evaluation plans, operating assumptions, and proposed revisions. A versioned input-output contract specifies required fields, evidence references, and stopping conditions. The contract makes the requested contribution and its required evidence explicit.

Five comparison conditions distinguish responsibility from work capacity. The full flower team represents all eight domains. An all-team omission removes one domain’s instructions wherever assigned but retains every contribution slot. A member-level withdrawal removes those instructions from one member while preserving colleagues’ overlapping assignments. Member removal withdraws the member and its scheduled contributions, reducing the default from forty calls to thirty-six. A single-agent condition assigns the full schedule to one generalist. Responsibility and single-agent comparisons hold case facts, learner and stakeholder constraints, stage order, model settings, call opportunities, and accepted-response character limits constant. Equal calls do not establish equal token use or elapsed time.

The interface shows assigned coverage, overlap, and scheduled calls before execution. Single-member withdrawal becomes distinct from all-team omission when scheduled colleagues share the domain. Shared understanding and learner and stakeholder responsibilities remain in every condition. Behavioral and coordination settings are declared assumptions, not

validated personality models. No rule forces omission to cause failure: another member may identify the issue, overlap may suffice, or the case may not need the contribution. Further studies should repeat runs and randomize condition order to examine responsibility, redundancy, and effort separately (Dede, 2026).

Assessment, Evidence, and Replay

The scorecard keeps three dimensions separate (Table 2). Nine criteria use evidence-anchored ratings from zero to four, averaged within each dimension on a 0–100 index. There is no overall total. Missing evidence remains unresolved, and criterion floors can expose essential failures hidden by an average. Flourishing scores concern support for people in the task; learning scores concern the design and available evidence. Neither measures observed human flourishing or learning gains.

Table 2. *Evidence-based scorecard dimensions*

Dimension	Criteria
Human flourishing support	Agency and dignity; access and inclusion; manageable burden
System performance	Technical correctness; reliability and recovery; implementation viability
Learning	Capability alignment; practice and feedback; independent assessment and transfer support

A packet with assignment metadata withheld supplies the task, artifact, rubric, and observations. The full-solution extension also includes the latest integrated record from each available stage, labeled as generated design records. Review guidance requires assessors to distinguish the retained executable Build from later plans and to give no implementation credit for a proposed repair. Raw conversations and assignment metadata remain outside the scoring packet. Ratings and evidence references are frozen before assignment reveal. Process review can then trace who raised a concern, how it entered a decision, and whether implementation or verification followed.

Masking does not remove every clue in an artifact, and model judgments can be biased (Zheng et al., 2023). The workflow supports human review, but the completed exploration used model-assisted assessment. Fresh runs expose variation in generated solutions; reviewer disagreement and context variation should be reported separately. Optional probability summaries propagate declared uncertainty and hypothetical profiles. They are conditional sensitivity analyses until calibrated against observations. Additional draws do not create additional teams or learners.

Each rating is accompanied by the cited evidence, uncertainty, and remaining questions. A working interface may still offer weak learning evidence or require an impractical delivery plan. A promising learning activity may fail a basic implementation check. Success criteria come from the task and evaluation protocol. A generalist or an unexpected design can satisfy those requirements without having a particular specialty represented on the team.

The retained browser harness executes the exact generated HTML with external traffic blocked. Its bounded checks cover differing task paths, correction, reset and reload, storage behavior, keyboard activation, and narrow-screen overflow. The records retain source and harness hashes, actions, errors, and observations. These checks establish what the software did under specified inputs. Human usability, accessibility, completion time, retention, and transfer require further evaluation with people.

Saved configurations, prompts, accepted responses, artifacts, failures, and event hashes support replay. Replaying the saved record and recomputing an assessment differ from requesting fresh stochastic responses. An evidence validator checks that review quotations occur in their cited sources. These tools allow a community member to inspect a claim, reproduce its supporting check, modify a configuration or artifact, and retain the new result as a distinct iteration.

The review interface accepts human judgments and imported model reviews. An organizer can prepare a packet with assignment metadata withheld, obtain criterion ratings with exact citations, freeze the result, and reveal the assignment afterward. Historical artifact reviews and new full-solution reviews have different scopes; the software rejects comparisons across them. Observations can be supplied programmatically. Participant data collection and calibration require a separate workflow. Practitioners can begin by assessing the retained examples and specifying the observations needed with actual users.

Worked Examples

The completed exploration contains eighteen runs: three fictional cases, three placements of learning-design instructions in an earlier fixed roster, and two repetitions. Each run covered eight stages and used twenty-one calls with gpt-6.1-sol at low reasoning effort. All 378 calls returned accepted responses. Eighteen artifact-scope reviews were frozen before assignment reveal; all 427 final evidence quotations matched their sources. Browser reports comprise sixteen completed reports and two partial reports caused by automation failures. These records remain separate from the new flower-domain preset and full-solution packet extension, which have not yet produced a new scored study.

Feedback conversation. Revising an initial-case response changed the retained answer to a later task while old feedback remained visible. The artifact could therefore present a contradictory answer-feedback pair. Implementation errors blocked the tested initial-practice path in two other candidates. The findings identify concrete implementation problems with consequences for the intended learning activity. A learner walkthrough should examine whether people understand which answer feedback refers to, notice an inconsistency, and recover without excessive effort.

Scientific reasoning. The member assigned measurement responsibilities identified that returning to the initial screen could overwrite the supposed pre-update response. The Instrument coordinator adopted the concern and proposed a Build revisit. Evaluate and Refine retained the issue and recommended holding real deployment. Browser inspection exposed the overwrite in the retained Build, which also offered a changed task involving a different confound. The task’s intended support for transfer coexisted with a defect in its evidence record. Follow-up should examine task quality with educators, preserve first attempts and assistance records, and assess learners on an independent delayed task.

Library access. A bounded digital path worked and the HTML included a printable paper route. Preparation, printing, human completion time, and helper demand remained unverified. During Refine, two members challenged the simulator’s assumption that every visitor required human help when the design allowed independent use. The coordinator concluded, “Optional support must not silently become a completion prerequisite.” This questioned the model’s assumptions. Observing staff preparation, help requests, queues, route switching, and recovery would test whether the proposed service could operate as intended.

Museum troubleshooting. A prospective fourth scenario asks volunteers to use observations to choose a diagnostic step or escalation. Changed faults and support availability

provide prospective probes of transfer and operating constraints. It supplies an extension point for the community; no generated results or ratings are reported for this case.

The earlier comparison produced different patterns across cases. Retaining learning-design instructions increased the raw learning rubric index over omission in both feedback repetitions (+50.0 and +16.7 points), while the science contrasts reversed (−8.3 and −16.7). Reinforcing science instructions increased the learning index while reducing the system index by 16.7 in both repetitions. These differences are on the 0–100 rubric scale, not modeled population outcomes. With two repetitions per condition, they identify examples to inspect rather than establish necessary human specialties. Implementation, integration, overlapping contributions, and variation remain possible explanations.

From Simulation to Real-World Inquiry

The worked examples give a starting point for empirical investigation. For each finding, a user can state an explanation, consider alternatives, identify the relevant people and setting, and specify what to observe next. This follows the distinction between how a design functions and how those functions produce learning (Sandoval, 2014). A software defect can be checked directly. Claims about comprehension, transfer, burden, or adoption require suitable observations with people in the intended setting.

The resources include configurable cases, executable artifacts, probe definitions, replay records, review tools, and questions for practitioners and learners. Users can change domain assignments or availability, introduce a challenge, inspect the resulting design, and revise it. A newly identified gap may require a different contribution, better information, or new software. The trace should show what changed and which observation would count as improvement.

Each case handoff identifies the artifact, retained probe, source evidence, proposed interpretation, and measurements still needed. A repaired artifact receives a new version and a new assessment set. Future field observations should record their context and procedure so that they can be compared with the earlier assumptions. An educator might challenge the task’s authenticity, a learner might reveal an unforeseen barrier, or a system owner might identify a missing dependency. Those findings can guide changes to the case, team, or design.

The completed exploration used fixed condition order and the same model for generation and assessment; two repetitions per cell cannot estimate stable failure probabilities. Six provider receipts were incomplete although application responses remained intact. Its scheduled Refine proposals did not rebuild the scored HTML. Assigning a domain does not establish proficiency, and removing instructions does not erase latent model knowledge. Simulated agents may also distort human behavior (Aher et al., 2023). No human learning outcomes or independent human ratings were collected. The simulation and its retained examples make proposed solutions open to inspection and help specify what needs to be tested with real teams, learners, and operating systems.

Acknowledgments. The authors acknowledge support from the LENS program.

Data availability. The community demonstration is available at https://capabilitymatters.org/agentic-le/, with flower-based configurations at https://capabilitymatters.org/agentic-le/experiments/. Source code, replay tools, and running instructions are maintained at https://github.com/wrgr/capabilitymatters. The earlier eighteen-run exploration records remain separate from the newly released flower framework.

Use of artificial intelligence. Artificial intelligence assisted software development, simulation, drafting, technical observation, and artifact assessment.

References

Aher, G. V., Arriaga, R. I., & Kalai, A. T. (2023). Using large language models to simulate multiple humans and replicate human subject studies. *Proceedings of Machine Learning Research, 202,* 337–371. https://proceedings.mlr.press/v202/aher23a.html

Ali, Z., Morehead, K., Jay, M., Arner, T., Lis, J., & Braitling, B. (2025). Learning engineering is a team sport: Collaborative explorations for team building strategies with short scenarios in various contexts. *International Consortium for Innovation and Collaboration in Learning Engineering (ICICLE) 2024 Conference Proceedings: Solving for Complexity at Scale*. https://doi.org/10.59668/2109.21837

Dede, C. (2026). What works when for whom under what conditions: Learning engineering as an enabler of component-based research. *Proceedings of the Learning Engineering Research Network Convening*. https://doi.org/10.59668/2551.25412

Epstein, J. M. (2008). Why model? *Journal of Artificial Societies and Social Simulation, 11*(4), Article 12. https://jasss.soc.surrey.ac.uk/11/4/12.html

Gröner, F., Verma, A., & Bronowitz, J. (2026). Automatically generating interactive learning experiences with an LLM-driven agentic pipeline. *Proceedings of the Learning Engineering Research Network Convening*. https://doi.org/10.59668/2551.25387

Ryng, H., & Suhr, D. (2026). Learning engineering by design: An agentic AI application for rapid, personalized health & safety training in disaster response and hazardous environments. *Proceedings of the Learning Engineering Research Network Convening*. https://doi.org/10.59668/2551.25171

Sandoval, W. (2014). Conjecture mapping: An approach to systematic educational design research. *Journal of the Learning Sciences, 23*(1), 18–36. https://doi.org/10.1080/10508406.2013.778204

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems, 36*. https://arxiv.org/abs/2306.05685